Guia de prompts para o Codex

Projeto: Desafio Renault 2026 · Conferência visual de recebimento
Para: Corrêa · **De:** Enzo Senatori · **Data:** 12/10/2026
Repositório: https://github.com/EnzoSenatori/DesafioRenault2026

Este guia traz os prompts, na ordem de uso, para continuar o projeto com o Codex. O contexto completo está no `CONTINUIDADE.md`, na raiz do repositório.

---

1. Antes de começar

1. Clone o repositório: `git clone https://github.com/EnzoSenatori/DesafioRenault2026.git`
2. Copie `.env.example` para `.env` e preencha `ROBOFLOW_API_KEY`. A chave não está no repositório: use a sua (site do Roboflow, *Settings* → *API Keys*) ou peça a do Enzo por um canal privado.
3. Abra a pasta do projeto no Codex.
4. Cole um prompt por vez. Espere o Codex terminar e revise o resultado antes de colar o próximo.

2. Ordem dos prompts

Ordem	Prompt	O que faz	O que precisa antes
1º	Inicial	Lê o projeto, cria o `AGENTS.md` e testa o ambiente	Repositório clonado e `.env` preenchido
2º	1	Avalia outros modelos de detecção	`gabarito.json` escrito e modelos candidatos escolhidos
3º	1b	Adota o modelo escolhido	Resultado do prompt 1 e escolha da equipe
4º	2	Modo câmera	Webcam
5º	3a ou 3b	Integração com o sistema do colega	Respostas às cinco perguntas da seção 6

Os passos 2 (câmera) e 3 (integração) não dependem do passo 1 e podem ser feitos em paralelo.

---

3. Prompt inicial — preparação

Cole este primeiro. Ele não muda o código, só prepara e confere o ambiente.

> Você vai dar continuidade a um projeto já em andamento. Antes de qualquer ação, leia por inteiro, nesta ordem: CONTINUIDADE.md, CLAUDE.md e README.md. O CONTINUIDADE.md é a referência principal: estado atual, decisões, contratos e os três próximos passos.
>
> Faça agora apenas a preparação (seção 7.1, prompt 0):
>
> 1. Crie um AGENTS.md na raiz com:
>   - as regras da seção 6 do CONTINUIDADE.md (convenções de código, escopo e armadilhas do Windows);
>   - os comandos para instalar, rodar a conferência e rodar os testes (seção 2);
>   - a regra de nunca imprimir, registrar ou gravar em arquivo a ROBOFLOW_API_KEY;
>   - a regra de não alterar o contrato do relatório (seção 3.6).
>   Não altere nenhum outro arquivo.
> 2. Confira o ambiente:
>   - se o .venv não existir, crie-o com Python 3.10 a 3.13 e instale o requirements.txt; se não houver Python compatível, pare e me avise;
>   - confirme que o .env existe e que a ROBOFLOW_API_KEY carrega, sem mostrar o valor;
>   - rode .venv\Scripts\python.exe -m pytest.
> 3. Rode uma conferência real:
>   .venv\Scripts\python.exe src\conferir.py --imagem "imagens/teste/Img 16.jpg" --nota notas/nota_divergente.json
>
> Ao terminar, pare e me entregue:
> - o conteúdo do AGENTS.md criado;
> - o resultado dos testes;
> - a saída da conferência (o esperado é DIVERGENTE, com falta de 1 chave inglesa);
> - qualquer problema de ambiente encontrado;
> - um resumo, em até 10 linhas, do que você entendeu sobre os três próximos passos e o que precisa de mim antes de começar cada um.
>
> Não comece nenhum dos três passos até eu pedir.

Resultado esperado: um `AGENTS.md` novo, os testes passando (39 ou mais) e a conferência terminando em DIVERGENTE.

---

4. Passo 1 — Melhorar a detecção

Tarefa humana antes do prompt (cerca de 15 minutos). Crie `imagens/teste/gabarito.json` com a contagem real de cada imagem, olhando as fotos. Conte só alicate (`Pliers`), chave de fenda (`Screwdriver`), chave inglesa (`Wrench`) e martelo (`Hammer`). Exemplo:

```
{
  "Img 15.jpg": {"Wrench": 4},
  "Img 16.jpg": {"Wrench": 1},
  "Img 7.jpeg": {"Hammer": 1, "Pliers": 3, "Screwdriver": 4}
}
```

Depois, procure em https://universe.roboflow.com modelos de detecção de ferramentas (buscas: "tools", "hand tools", "wrench hammer pliers screwdriver") e anote o `model_id` (`projeto/versão`) e as classes de cada um.

Prompt 1 — avaliação de modelos

Troque `<lista de model_id>` pelos modelos que você escolheu.

> Leia AGENTS.md e as seções 3, 4 e 7.2 do CONTINUIDADE.md. Crie src/avaliar_modelos.py, que recebe uma lista de model_id por --modelos e, opcionalmente, um mapeamento de classes por modelo (JSON) para as nossas classes Pliers, Screwdriver, Wrench e Hammer.
>
> Para cada modelo e cada imagem de imagens/teste/, o script chama o Roboflow reaproveitando detector.criar_cliente e detector.converter_resposta (troque só o model_id numa cópia do config). Ele conta as detecções com confiança de pelo menos 0,5 e compara com imagens/teste/gabarito.json.
>
> Gere saidas/avaliacao/<modelo>.json e imprima uma tabela por modelo com: imagens com contagem exata, total de itens detectados corretamente, faltas e excessos. Salve uma resposta bruta de cada modelo em fixtures/ antes de assumir o formato. Não altere config.json nem o contrato do relatório. Inclua testes sem rede para a lógica de comparação com o gabarito.
>
> Modelos a avaliar: <lista de model_id>, além do atual tools-classification-byznk/2 como referência.

Prompt 1b — adotar o modelo escolhido

Use depois que a equipe escolher o modelo. Troque `<model_id>` e `<imagens>`.

> Troque o modelo para <model_id> no config.json e ajuste de_para, nomes_exibicao e classes_ignoradas às classes dele. Rode src/testar_modelo.py e me mostre um resumo por imagem.
>
> Depois, com as imagens <imagens>, reescreva notas/nota_conforme.json e notas/nota_divergente.json (a divergente declara uma unidade a mais do que a imagem mostra), regenere as evidências de docs/ e atualize as seções de limitações do README.md e do CONTINUIDADE.md. Dê crédito ao autor do novo modelo no README, conforme a licença dele.

Se nenhum modelo servir: não peça ao Codex para treinar um modelo. Leve a decisão à equipe, porque treino está fora de escopo.

---

5. Passo 2 — Modo câmera

Só faz sentido com webcam. Sem ferramentas físicas, dá para testar apontando a câmera para fotos de ferramentas na tela de um celular.

Prompt 2 — modo câmera

> Leia AGENTS.md e as seções 3 e 7.3 do CONTINUIDADE.md. Implemente src/camera.py conforme os requisitos da seção 7.3, reaproveitando conferir_quadro, montar_evidencia e salvar_jpg, sem duplicar lógica.
>
> Mantenha a lógica testável fora do laço de vídeo e escreva tests/test_camera.py sem rede e sem webcam (use quadros numpy sintéticos e um cliente falso). Atualize o README com uma seção "Modo câmera". Rode os testes e me mostre o resultado.

Para testar depois: `.venv\Scripts\python.exe src\camera.py --nota notas/nota_divergente.json`. A tecla espaço captura o quadro e Q sai.

---

6. Passo 3 — Integração

Antes do prompt, responda com o colega:

1. Em que linguagem e ambiente o outro sistema roda? Na mesma máquina?
2. Como a imagem chega: caminho de arquivo, bytes, URL ou câmera?

3. Como a nota chega: arquivo JSON, objeto em memória ou outro formato, como XML de NF-e?
4. O que o orquestrador precisa de volta: só o status, o relatório inteiro ou também a imagem de evidência?
5. A chamada é síncrona (espera a resposta) ou por fila ou pasta?

Se os dois sistemas rodam na mesma máquina, use o **prompt 3a**. Se estão em máquinas diferentes ou se comunicam pela web, use o **prompt 3b**, mas só com o aval da equipe, porque uma API web estava fora de escopo.

Prompt 3a — integração local

Troque o trecho entre `< >` pela descrição do outro sistema, usando as respostas acima.

> Leia AGENTS.md e as seções 3 e 7.4 do CONTINUIDADE.md. O outro sistema é <descreva: linguagem, como envia imagem e nota, o que espera de volta>.
>
> Crie o ponto de integração usando conferir_quadro ou a CLI com --json, sem mudar o contrato do relatório. Se a nota vier em outro formato, crie src/conversor_nota.py, que a converte para o formato da seção 3.3, com testes. Escreva um teste de integração com cliente falso cobrindo os três status (conforme, divergente, erro).

Prompt 3b — API HTTP (só com o aval da equipe)

> Leia AGENTS.md e as seções 3 e 7.4 do CONTINUIDADE.md. Crie src/api.py com FastAPI e um endpoint POST /conferir que recebe uma imagem (multipart) e a nota (JSON) e devolve o relatório da seção 3.6, com a evidência em base64 num campo opcional imagem_anotada_base64.
>
> Reaproveite validar_nota e conferir_quadro. Nota inválida responde HTTP 422 com a mensagem; falha técnica responde 200 com status "erro", como no restante do sistema. Fixe as versões no requirements.txt depois de instalar. Escreva testes com o TestClient do FastAPI e um cliente falso do Roboflow. Documente no README como subir a API.

---

7. Dicas para trabalhar com o Codex

- **Um passo por vez:** cole um prompt, revise o que mudou e rode os testes antes do próximo.
- **Desconfie de resultados sem execução:** se o Codex afirmar algo sobre o modelo ou a resposta do Roboflow sem ter rodado, peça que rode e mostre a saída.
- **Chave fora do repositório:** a `ROBOFLOW_API_KEY` nunca deve aparecer em código, log ou commit. Antes de cada commit, confira com `git status` que o `.env` não está na lista.
- **Contrato fixo:** o formato do `relatorio.json` (seção 3.6 do `CONTINUIDADE.md`) é o que a integração consome. Campos novos são permitidos; renomear ou remover campos não é.
- **Codex parado ou confuso:** peça para ele reler o `AGENTS.md` e a seção do passo atual no `CONTINUIDADE.md`.
- **Ao final de cada passo:** atualize o checklist da seção 8 do `CONTINUIDADE.md` e faça o commit.